

AI Incident Database Datathon Report

Generative AI Risks and Safeguards for SEC Victoria

Table of contents

1	Project Setup	2
2	Data Source and Collection Structure	3
2.1	Connect to MongoDB	3
2.2	Collection Sizes	3
2.3	Report Characteristics	4
2.3.1	Most Common Report Languages	5
2.4	Reports Over Time	6
3	Data Quality and Preparation	8
3.1	Reports per Incident	8
3.2	Identifier Uniqueness	10
3.3	Create Reduced Working Datasets	12
3.4	Missing-Value Checks	12
4	Joining Reports to Incidents	14
4.1	MongoDB Join	14
4.2	Validate the Join	14
4.3	Reports Linked to Multiple Incidents	15
4.4	Cross-Check Incident Report Counts	16
5	Reproducibility and Transformation Checks	17
5.1	Date and Scope Audit	18
5.2	Source-Domain Distribution	18
5.3	Server-Side and R-Side Join Validation	20
5.4	Text-Cleaning Audit	21
6	Defining the Generative AI Analysis Population	21
6.1	Build One Text Record per Incident	22

6.2	Identify Generative AI Incidents	23
7	Exploratory Text Analysis	25
7.1	Prepare Linked Text	26
7.2	Tokenisation and Cleaning	26
7.3	Incident-Level Word Prevalence	26
7.4	Bigram Analysis	28
8	Generative AI Failure Patterns	29
8.1	Classify Failure Patterns	29
8.2	Prevalence of Failure Patterns	33
8.3	Number of Failure Patterns per Incident	37
9	Root-Cause and Control-Gap Patterns	40
9.1	Classify Control-Gap Indicators	40
9.2	Quantify Control Gaps	44
10	Implications for SEC Victoria	46
10.1	Retail Operations	46
10.2	Wholesale Operations	47
10.3	Household Electrification	48
11	Recommended Safeguards	49
11.1	Recommended Rollout Model	51
11.1.1	Tier 1: Low-consequence assistance	52
11.1.2	Tier 2: Customer or analytical decision support	52
11.1.3	Tier 3: High-consequence or operational actions	52
12	Key Findings	53
13	Limitations and Ethical Considerations	53
14	Conclusion	54

1 Project Setup

This report analyses the AI Incident Database (AIID) to identify recurring generative AI failure patterns and assess what those patterns could mean for SEC Victoria if internal generative AI tools were deployed across retail, wholesale and household-electrification operations.

The analysis follows four questions:

1. What failure patterns repeatedly appear in generative AI incidents?
2. How common are those patterns across distinct incidents?

3. Which of those risks are most relevant to SEC Victoria’s proposed internal uses?
4. What practical safeguards are supported by the incident evidence?

2 Data Source and Collection Structure

The analysis uses the AI Incident Database collections for **reports** and **incidents**.

A report represents a source document describing an AI-related event, while an incident represents the underlying event recorded by AIID. Multiple reports may therefore describe the same incident.

This distinction is important for the SEC Victoria question because counting reports alone could make highly publicised failures appear more common than failures that received less media attention.

2.1 Connect to MongoDB

2.2 Collection Sizes

```
n_reports <- reports_conn$count()
n_incidents <- incidents_conn$count()

collection_sizes <- tibble(
  Collection = c("Reports", "Incidents"),
  Records = c(n_reports, n_incidents)
)

knitr::kable(
  collection_sizes,
  caption = "Size of the two AIID collections used in the analysis."
)
```

Table 1: Size of the two AIID collections used in the analysis.

Collection	Records
Reports	7526
Incidents	1632

These two collections serve different purposes. Reports provide detailed source text, while incidents provide the appropriate unit for estimating how widely a failure pattern occurs.

2.3 Report Characteristics

The database contains reports in multiple languages and includes an `is_incident_report` classification. These fields are inspected for data-quality purposes rather than used as automatic exclusion rules.

```
n_reports_en <- reports_conn$count(
  '{"language": "en"}'
)

n_incident_reports <- reports_conn$count(
  '{"is_incident_report": true}'
)

n_reports_en_incident <- reports_conn$count(
  '{
    "language": "en",
    "is_incident_report": true
  }'
)

report_characteristics <- tibble(
  Measure = c(
    "English-language reports",
    "Reports marked as incident reports",
    "English reports marked as incident reports"
  ),
  Reports = c(
    n_reports_en,
    n_incident_reports,
    n_reports_en_incident
  )
)

knitr::kable(
  report_characteristics,
  caption = "Selected characteristics of the AIID reports collection."
)
```

Table 2: Selected characteristics of the AIID reports collection.

Measure	Reports
English-language reports	7270
Reports marked as incident reports	6301
English reports marked as incident reports	6053

2.3.1 Most Common Report Languages

```
reports_by_language <- reports_conn$aggregate('[
  {
    "$group": {
      "_id": "$language",
      "n_reports": { "$sum": 1 }
    }
  },
  {
    "$sort": {
      "n_reports": -1
    }
  }
]') |>
as_tibble() |>
rename(
  language = `_id`
) |>
mutate(
  language = replace_na(language, "Missing")
)

reports_by_language |>
slice_head(n = 10) |>
rename(
  Language = language,
  Reports = n_reports
) |>
knitr::kable(
  caption = "Ten most common language labels in the reports collection."
)
```

Table 3: Ten most common language labels in the reports collection.

Language	Reports
en	7270
es	61
it	26
de	22
zh-CN	21
fr	20
pt	10
sq	10
ja	8
ko	7

No language filter is applied to the main dataset. However, later keyword-based text classification is primarily English-focused, which is treated as a limitation rather than silently removing non-English observations.

2.4 Reports Over Time

```
reports_by_year <- reports_conn$aggregate('[
{
  "$match": {
    "date_published": { "$ne": null }
  }
},
{
  "$group": {
    "_id": {
      "$year": "$date_published"
    },
    "n_reports": { "$sum": 1 }
  }
},
{
  "$sort": {
    "_id": 1
  }
}
]') |>
```

```
as_tibble() |>
rename(
  year = `_id`
)
```

```
reports_by_year |>
  filter(
    year >= 1967,
    year <= year(analysis_end)
  ) |>
  ggplot(
    aes(
      x = year,
      y = n_reports
    )
  ) +
  geom_col() +
  scale_x_continuous(
    breaks = seq(1970, 2025, by = 5)
  ) +
  labs(
    x = "Publication year",
    y = "Number of reports"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
)
```

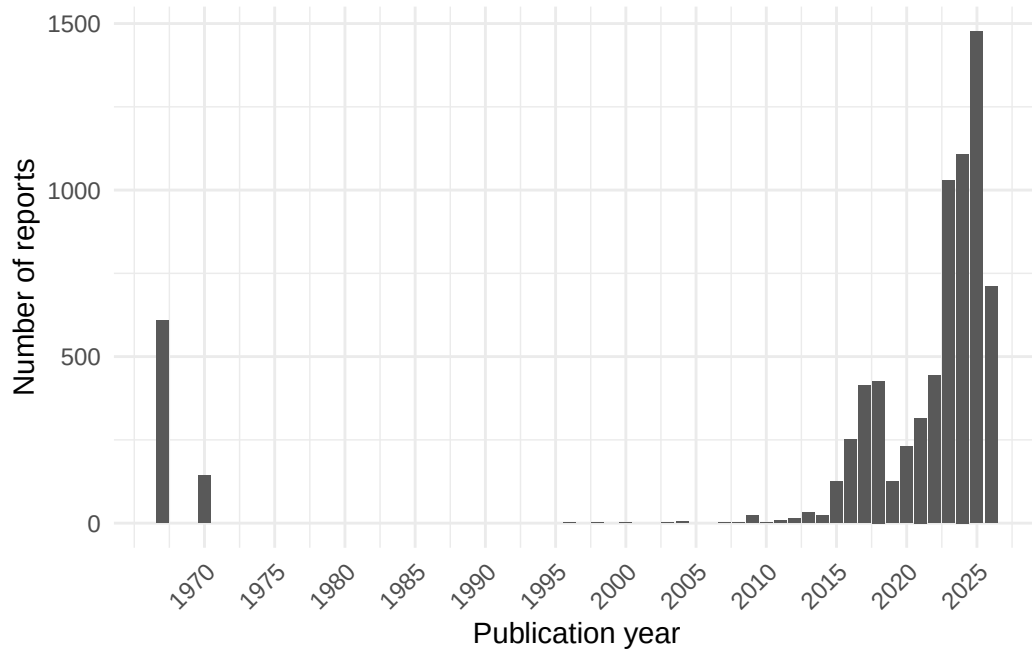


Figure 1: Number of AIID reports by publication year.

The series describes publication activity rather than the exact timing of the underlying incidents. A later publication can describe an earlier event.

3 Data Quality and Preparation

3.1 Reports per Incident

One incident can be supported by many separate reports. This creates an important analytical issue: counting words or topics at report level can give highly reported incidents disproportionate influence.

```
reports_per_incident <- incidents_conn$aggregate('[
{
  "$project": {
    "_id": 0,
    "incident_id": 1,
    "title": 1,
    "n_reports": {
      "$size": {
```



```

      "$ifNull": ["$reports", []]
    }
  }
},
{
  "$sort": {
    "n_reports": -1
  }
}
]') |>
  as_tibble()

```

```

reports_per_incident_summary <- reports_per_incident |>
  summarise(
    Incidents = n(),
    `Median reports` = median(n_reports),
    `Mean reports` = round(mean(n_reports), 2),
    `Maximum reports` = max(n_reports)
  )

knitr::kable(
  reports_per_incident_summary,
  caption = "Number of reports linked to each AIID incident."
)

```

Table 4: Number of reports linked to each AIID incident.

Incidents	Median reports	Mean reports	Maximum reports
1632	2	3.94	58

```

reports_per_incident |>
  count(n_reports) |>
  filter(n_reports <= 20) |>
  ggplot(
    aes(
      x = n_reports,
      y = n
    )
  ) +

```

```
geom_col() +
labs(
  x = "Reports linked to incident",
  y = "Number of incidents"
) +
theme_minimal()
```

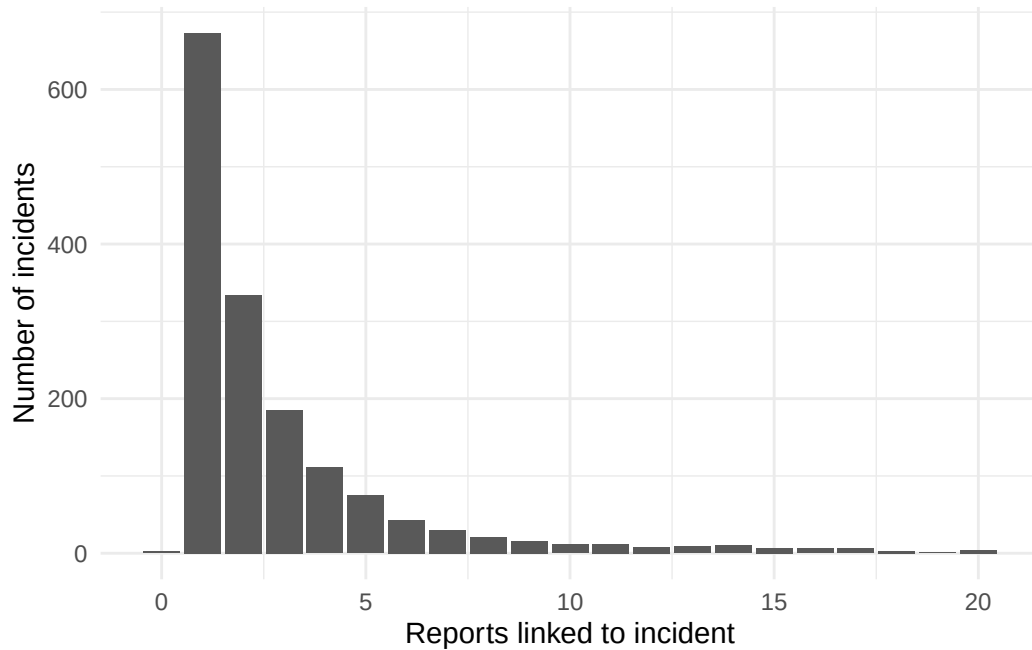


Figure 2: Distribution of the number of source reports linked to each incident.

The figure is restricted to incidents with up to 20 reports so that the main distribution remains readable. Incidents with larger numbers of reports are retained in the analysis.

Because reporting intensity is uneven, the main risk findings later in this report are calculated using **distinct incidents**, not raw report frequencies.

3.2 Identifier Uniqueness

```
reports_id_check <- reports_conn$aggregate('[
{
  "$group": {
```

```

      "_id": "$report_number",
      "n": { "$sum": 1 }
    }
  },
  {
    "$match": {
      "n": { "$gt": 1 }
    }
  }
]') |>
  as_tibble()

incidents_id_check <- incidents_conn$aggregate('[
  {
    "$group": {
      "_id": "$incident_id",
      "n": { "$sum": 1 }
    }
  },
  {
    "$match": {
      "n": { "$gt": 1 }
    }
  }
]') |>
  as_tibble()

identifier_check <- tibble(
  Identifier = c(
    "report_number",
    "incident_id"
  ),
  `Duplicate values` = c(
    nrow(reports_id_check),
    nrow(incidents_id_check)
  )
)

knitr::kable(
  identifier_check,
  caption = "Duplicate identifier checks."
)

```

Table 5: Duplicate identifier checks.

Identifier	Duplicate values
report_number	0
incident_id	0

A result of zero indicates that the identifier is unique within its collection.

3.3 Create Reduced Working Datasets

Only fields required for quality checks, joins and the later incident analysis are extracted.

3.4 Missing-Value Checks

```
report_missingness <- reports_filtered |>
  summarise(
    `Report number` = sum(is.na(report_number)),
    Title = sum(is.na(title) | trimws(title) == ""),
    Description = sum(is.na(description) | trimws(description) == ""),
    `Plain text` = sum(is.na(plain_text) | trimws(plain_text) == ""),
    Language = sum(is.na(language) | trimws(language) == ""),
    `Publication date` = sum(is.na(date_published)),
    `Source domain` = sum(is.na(source_domain) | trimws(source_domain) == ""),
    URL = sum(is.na(url) | trimws(url) == "")
  ) |>
  pivot_longer(
    everything(),
    names_to = "Field",
    values_to = "Missing"
  )

knitr::kable(
  report_missingness,
  caption = "Missing values in selected report fields."
)
```

Table 6: Missing values in selected report fields.

Field	Missing
Report number	0
Title	807
Description	5513
Plain text	807
Language	0
Publication date	0
Source domain	807
URL	807

```
incident_missingness <- incidents_filtered |>
  summarise(
    `Incident ID` = sum(is.na(incident_id)),
    Title = sum(is.na(title) | trimws(title) == ""),
    Description = sum(is.na(description) | trimws(description) == ""),
    Date = sum(is.na(date) | trimws(date) == ""),
    `No linked reports` = sum(lengths(reports) == 0),
    `No implicated system` = sum(lengths(implicated_systems) == 0),
    `No alleged deployer` =
      sum(lengths(`Alleged deployer of AI system`) == 0),
    `No alleged developer` =
      sum(lengths(`Alleged developer of AI system`) == 0),
    `No harmed party` =
      sum(lengths(`Alleged harmed or nearly harmed parties`) == 0)
  ) |>
  pivot_longer(
    everything(),
    names_to = "Field",
    values_to = "Missing"
  )

knitr::kable(
  incident_missingness,
  caption = "Missing values in selected incident fields."
)
```

Table 7: Missing values in selected incident fields.

Field	Missing
Incident ID	0
Title	0
Description	0
Date	0
No linked reports	2
No implicated system	686
No alleged deployer	0
No alleged developer	0
No harmed party	1

These checks determine which fields can support later conclusions. Missing data are not automatically interpreted as evidence that a characteristic was absent from an incident.

4 Joining Reports to Incidents

The `incidents.reports` field stores report identifiers. Reports are therefore joined to incidents by matching `reports.report_number` to the report identifiers stored within each incident.

4.1 MongoDB Join

A row in this dataset represents a **report-incident relationship**, not necessarily a unique report or a unique incident.

4.2 Validate the Join

```
link_summary <- reports_incidents_joined |>
  summarise(
    `Joined rows` = n(),
    `Unique reports` = n_distinct(report_number),
    `Unique linked incidents` =
      n_distinct(incident_id, na.rm = TRUE),
    `Reports without matched incident` =
      sum(is.na(incident_id))
  ) |>
  pivot_longer(
```

```

    everything(),
    names_to = "Measure",
    values_to = "Value"
  )

knitr::kable(
  link_summary,
  caption = "Summary of the report-incident join."
)

```

Table 8: Summary of the report-incident join.

Measure	Value
Joined rows	7684
Unique reports	7526
Unique linked incidents	1630
Reports without matched incident	1252

4.3 Reports Linked to Multiple Incidents

```

reports_multiple_incidents <- reports_incidents_joined |>
  filter(!is.na(incident_id)) |>
  group_by(report_number) |>
  summarise(
    n_incidents = n_distinct(incident_id),
    .groups = "drop"
  ) |>
  filter(n_incidents > 1) |>
  arrange(desc(n_incidents))

multiple_link_summary <- tibble(
  Measure = c(
    "Reports linked to more than one incident",
    "Maximum incidents linked to one report"
  ),
  Value = c(
    nrow(reports_multiple_incidents),
    ifelse(
      nrow(reports_multiple_incidents) == 0,

```

```

    1,
    max(reports_multiple_incidents$n_incidents)
  )
)
)

knitr::kable(
  multiple_link_summary,
  caption = "Reports linked to multiple AIID incidents."
)

```

Table 9: Reports linked to multiple AIID incidents.

Measure	Value
Reports linked to more than one incident	110
Maximum incidents linked to one report	8

4.4 Cross-Check Incident Report Counts

```

reports_per_incident_joined <- reports_incidents_joined |>
  filter(!is.na(incident_id)) |>
  group_by(incident_id) |>
  summarise(
    n_reports_joined = n_distinct(report_number),
    .groups = "drop"
  )

report_count_check <- reports_per_incident |>
  select(
    incident_id,
    n_reports_original = n_reports
  ) |>
  left_join(
    reports_per_incident_joined,
    by = "incident_id"
  ) |>
  mutate(
    n_reports_joined = coalesce(n_reports_joined, 0L),
    difference =

```



```

      n_reports_original - n_reports_joined
    ) |>
    filter(difference != 0)

join_validation <- tibble(
  Measure = c(
    "Incidents with different reconstructed report counts",
    "Join validation result"
  ),
  Result = c(
    as.character(nrow(report_count_check)),
    ifelse(
      nrow(report_count_check) == 0,
      "No discrepancies detected",
      "Discrepancies detected"
    )
  )
)

knitr::kable(
  join_validation,
  caption = "Validation of report counts reconstructed through the join."
)

```

Table 10: Validation of report counts reconstructed through the join.

Measure	Result
Incidents with different reconstructed report counts	0
Join validation result	No discrepancies detected

This validation reduces the risk that later failure-pattern estimates are driven by an incorrect report-incident relationship.

5 Reproducibility and Transformation Checks

Sections of the team workflow also independently validate date handling, server-side transformation, joining and text cleaning.

5.1 Date and Scope Audit

```
knitr::kable(  
  s679_audit_table("date_audit"),  
  caption = "Audit of publication-date fields used in the server-side workflow."  
)
```

Table 11: Audit of publication-date fields used in the server-side workflow.

Audit measure	Result	n
date	TRUE	7526

The server-side workflow uses a half-open date interval so that the end boundary is unambiguous.

5.2 Source-Domain Distribution

```
s679_domains |>  
  arrange(desc(n_reports)) |>  
  slice_head(n = 10) |>  
  rename(  
    `Source domain` = source_domain,  
    Reports = n_reports  
  ) |>  
  knitr::kable(  
    caption = "Most common source domains in the server-side analytical window."  
  )
```

Table 12: Most common source domains in the server-side analytical window.

Source domain	Reports	denominator_reports	percent_of_all_reports
nytimes.com	199	5315	3.744120
theguardian.com	162	5315	3.047977
washingtonpost.com	158	5315	2.972719
reuters.com	115	5315	2.163688
theverge.com	93	5315	1.749765
wired.com	75	5315	1.411101

Source domain	Reports	denominator_reports	percent_of_all_reports
bbc.com	72	5315	1.354657
futurism.com	72	5315	1.354657
arstechnica.com	66	5315	1.241769
apnews.com	65	5315	1.222954

```
s679_domains |>
  arrange(desc(n_reports)) |>
  slice_head(n = 15) |>
  mutate(
    source_domain =
      forcats::fct_reorder(
        source_domain,
        n_reports
      )
  ) |>
  ggplot(
    aes(
      x = source_domain,
      y = n_reports
    )
  ) +
  geom_col() +
  coord_flip() +
  labs(
    x = NULL,
    y = "Number of reports"
  ) +
  theme_minimal()
```

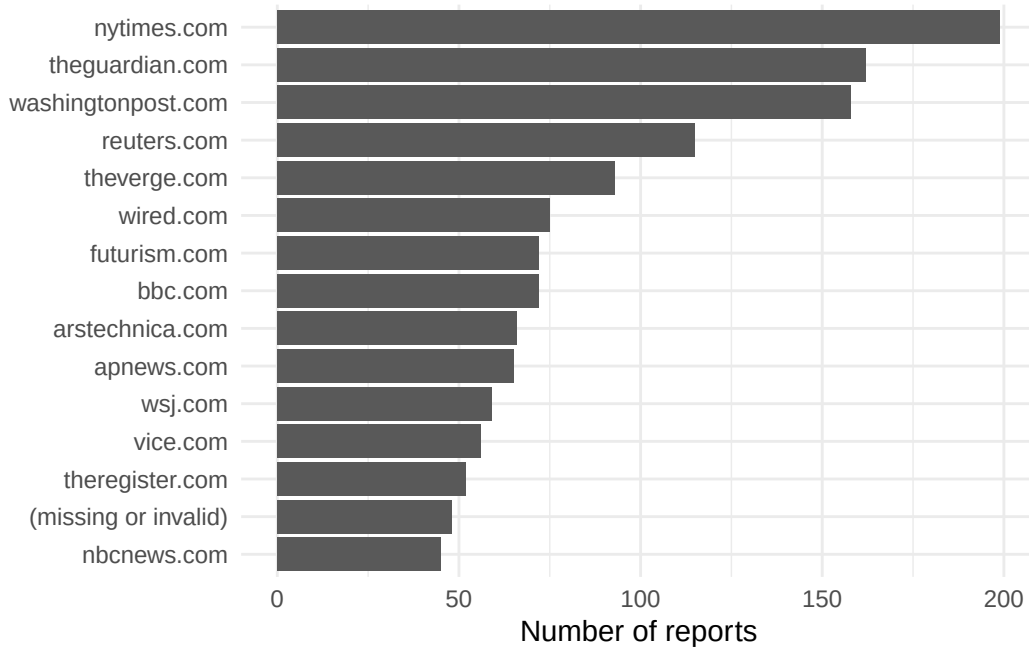


Figure 3: Most common source domains in the server-side analytical window.

Source domains describe where reports were published. They are not treated as independent incidents or as measures of incident severity.

5.3 Server-Side and R-Side Join Validation

```
knitr::kable(
  s679_audit_table("join_audit"),
  caption = "Independent validation of server-side and R-side joins."
)
```

Table 13: Independent validation of server-side and R-side joins.

Audit measure	Result
Eligible reports	3087
Matched reports	2848
Unmatched reports	239
Reports linked to multiple incidents	78
Matched-report percent	92.26%

Audit measure	Result
Joined rows	3201

Agreement is tested using report-incident identifier pairs rather than relying only on equal row counts.

5.4 Text-Cleaning Audit

```
knitr::kable(
  s679_audit_table("cleaning_audit"),
  caption = "Quality checks applied to cleaned report text."
)
```

Table 14: Quality checks applied to cleaned report text.

Audit measure	Result
Reports selected	3087
Missing or non-string original bodies	0
Original bodies empty/whitespace only	45
Usable cleaned bodies	3042
Non-empty originals becoming empty	0
Exact-duplicate cleaned-body groups	34

Text cleaning is intentionally conservative. Original text is retained and cleaned text is stored separately so transformations can be audited.

6 Defining the Generative AI Analysis Population

The challenge concerns generative AI rather than every form of artificial intelligence represented in AIID.

A reproducible indicator is therefore used to identify incidents whose titles, descriptions or associated reports contain clear generative-AI terminology or major generative-AI system names.

This is a screening method rather than an official AIID taxonomy.

6.1 Build One Text Record per Incident

```
incident_analysis_text <- reports_incidents_joined |>
  filter(
    !is.na(incident_id)
  ) |>
  group_by(
    incident_id,
    incident_title,
    incident_description,
    incident_date
  ) |>
  summarise(
    n_linked_reports =
      n_distinct(report_number),

    report_languages =
      str_c(
        sort(
          unique(
            na.omit(language)
          )
        ),
        collapse = ", "
      ),

    report_text =
      str_c(
        unique(
          plain_text[
            !is.na(plain_text) &
            str_trim(plain_text) != ""
          ]
        ),
        collapse = " "
      ),

    .groups = "drop"
  ) |>
  mutate(
    analysis_text =
      str_to_lower(
```

```

str_c(
  coalesce(
    incident_title,
    ""
  ),
  coalesce(
    incident_description,
    ""
  ),
  coalesce(
    report_text,
    ""
  ),
  sep = " "
)
)
)

```

Working at incident level prevents incidents with many media reports from automatically receiving more weight.

6.2 Identify Generative AI Incidents

```

genai_pattern <- regex(
  paste(
    c(
      "generative ai",
      "generative artificial intelligence",
      "large language model",
      "large language models",
      "\\bllm\\b",
      "\\bllms\\b",
      "chatgpt",
      "openai",
      "gpt-3",
      "gpt-4",
      "gpt-5",
      "gemini",
      "bard",
      "claude",
    )
  )
)

```

```

      "copilot",
      "dall-e",
      "dall·e",
      "midjourney",
      "stable diffusion",
      "text generator",
      "image generator",
      "generative model",
      "generative models"
    ),
    collapse = "|"
  ),
  ignore_case = TRUE
)

incident_analysis_text <- incident_analysis_text |>
  mutate(
    genai_indicator =
      str_detect(
        analysis_text,
        genai_pattern
      )
  )

genai_incidents <- incident_analysis_text |>
  filter(genai_indicator)

n_genai_incidents <- nrow(genai_incidents)

```

```

genai_scope <- tibble(
  Measure = c(
    "Total AIID incidents",
    "Incidents with linked report text",
    "Incidents identified by the GenAI screen",
    "GenAI screen share of all incidents"
  ),
  Result = c(
    format(
      n_incidents,
      big.mark = ",",
    ),
    format(

```



```

    n_distinct(
      incident_analysis_text$incident_id
    ),
    big.mark = ",",
  ),
  format(
    n_genai_incidents,
    big.mark = ",",
  ),
  pct(
    100 *
      n_genai_incidents /
      n_incidents
  )
)
)

knitr::kable(
  genai_scope,
  caption = "Scope of the generative AI incident screen."
)

```

Table 15: Scope of the generative AI incident screen.

Measure	Result
Total AIID incidents	1,632
Incidents with linked report text	1,630
Incidents identified by the GenAI screen	598
GenAI screen share of all incidents	36.6%

The GenAI screen is intentionally reproducible and conservative, but it can miss incidents that involve generative AI without using the selected terminology.

7 Exploratory Text Analysis

Text analysis is used to identify recurring language and help develop interpretable failure categories. Word frequencies are treated as exploratory evidence rather than as the final measure of risk.

7.1 Prepare Linked Text

```
text_data <- reports_incidents_joined |>
  filter(
    !is.na(incident_id),
    !is.na(plain_text),
    str_trim(plain_text) != ""
  ) |>
  select(
    report_number,
    incident_id,
    incident_title,
    language,
    plain_text
  )
```

7.2 Tokenisation and Cleaning

7.3 Incident-Level Word Prevalence

```
n_incidents_text <-
  n_distinct(
    text_data$incident_id
  )

incident_word_prevalence <- tokens_risk |>
  distinct(
    incident_id,
    word
  ) |>
  count(
    word,
    sort = TRUE
  ) |>
  mutate(
    pct_incidents =
      100 *
      n /
      n_incidents_text
  )
```

```

incident_word_prevalence |>
  slice_head(n = 20) |>
  rename(
    Word = word,
    Incidents = n,
    `Incident prevalence (%)` =
      pct_incidents
  ) |>
  mutate(
    `Incident prevalence (%)` =
      round(
        `Incident prevalence (%)`,
        1
      )
  ) |>
  knitr::kable(
    caption = "Common words measured by distinct incident prevalence."
  )

```

Table 16: Common words measured by distinct incident prevalence.

Word	Incidents	Incident prevalence (%)
found	941	57.7
information	938	57.5
social	907	55.6
public	897	55.0
real	894	54.8
based	882	54.1
called	871	53.4
online	853	52.3
generated	809	49.6
data	781	47.9
reported	781	47.9
users	778	47.7
similar	764	46.9
video	756	46.4
human	734	45.0
content	732	44.9
news	730	44.8
day	727	44.6
added	707	43.4

Word	Incidents	Incident prevalence (%)
tools	700	42.9

This measure counts an incident once regardless of how many times a word appears in reports about that incident.

7.4 Bigram Analysis

```
incident_bigram_prevalence |>
  select(
    phrase,
    n,
    pct_incidents
  ) |>
  slice_head(n = 15) |>
  rename(
    Phrase = phrase,
    Incidents = n,
    `Incident prevalence (%)` =
      pct_incidents
  ) |>
  mutate(
    `Incident prevalence (%)` =
      round(
        `Incident prevalence (%)`,
        1
      )
  ) |>
  knitr::kable(
    caption = "Common two-word phrases measured by distinct incident prevalence."
  )
```

Table 17: Common two-word phrases measured by distinct incident prevalence.

Phrase	Incidents	Incident prevalence (%)
ai generated	590	36.2
ai tools	301	18.5
generative ai	283	17.4

Phrase	Incidents	Incident prevalence (%)
law enforcement	271	16.6
machine learning	223	13.7
real world	221	13.6
ai powered	215	13.2
generated content	183	11.2
san francisco	183	11.2
york times	183	11.2
elon musk	164	10.1
immediately respond	160	9.8
ai models	158	9.7
chief executive	156	9.6
personal information	156	9.6

The exploratory text results help reveal recurring themes, but safeguards should not be based simply on the frequency of individual words. The next section therefore classifies interpretable failure patterns at incident level.

8 Generative AI Failure Patterns

The central analysis classifies each identified GenAI incident against a set of recurring failure patterns.

The categories are not official AIID labels. They are transparent text-based indicators designed to measure whether the incident evidence contains language associated with a particular type of failure.

An incident can belong to more than one category.

8.1 Classify Failure Patterns

```
genai_risks <- genai_incidents |>
  mutate(

    inaccurate_output =
      str_detect(
        analysis_text,
        regex(
          paste(
```

```

        c(
            "hallucinat",
            "fabricat",
            "made up",
            "false information",
            "incorrect information",
            "inaccurate information",
            "false claim",
            "wrong answer",
            "incorrect answer",
            "misinformation",
            "invented",
            "nonexistent"
        ),
        collapse = "|"
    ),
    ignore_case = TRUE
)
),

unsafe_advice =
str_detect(
    analysis_text,
    regex(
        paste(
            c(
                "harmful advice",
                "unsafe advice",
                "dangerous advice",
                "unsafe recommendation",
                "harmful recommendation",
                "self-harm",
                "suicide advice",
                "medical advice",
                "legal advice"
            ),
            collapse = "|"
        ),
        ignore_case = TRUE
    )
),

```

```

privacy_data =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "privacy",
          "personal data",
          "personal information",
          "confidential information",
          "confidential data",
          "sensitive information",
          "sensitive data",
          "data leak",
          "data breach",
          "exposed data",
          "training data"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  ),

bias_discrimination =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "\\bbias\\b",
          "\\bbiased\\b",
          "discriminat",
          "racial bias",
          "gender bias",
          "stereotyp",
          "unequal treatment",
          "unfair treatment"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  )

```

```

    )
),

fraud_impersonation =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "\\bfraud\\b",
          "\\bscam\\b",
          "\\bscams\\b",
          "impersonat",
          "deepfake",
          "phishing",
          "identity theft",
          "fake voice",
          "fake image",
          "fake video",
          "deceptive content"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  ),

automation_oversight =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "automated decision",
          "automatic decision",
          "without human",
          "human oversight",
          "human review",
          "lack of oversight",
          "no oversight",
          "autonomous decision",
          "automated action"
        )
      )
    )
  )

```



```

    ),
    collapse = "|"
  ),
  ignore_case = TRUE
)
),

security_misuse =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "prompt injection",
          "jailbreak",
          "jailbreaking",
          "security vulnerability",
          "cybersecurity",
          "\\bmalware\\b",
          "\\bhack",
          "unauthorised access",
          "unauthorized access",
          "security flaw",
          "security breach"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  )
)

```

8.2 Prevalence of Failure Patterns

```

risk_prevalence <- genai_risks |>
  summarise(
    `Incorrect or fabricated output` =
      sum(
        inaccurate_output,
        na.rm = TRUE
      )
  )

```

```

    ),

    `Unsafe or harmful advice` =
      sum(
        unsafe_advice,
        na.rm = TRUE
      ),

    `Privacy or confidential-data risk` =
      sum(
        privacy_data,
        na.rm = TRUE
      ),

    `Bias or discrimination` =
      sum(
        bias_discrimination,
        na.rm = TRUE
      ),

    `Fraud or impersonation` =
      sum(
        fraud_impersonation,
        na.rm = TRUE
      ),

    `Automation or oversight failure` =
      sum(
        automation_oversight,
        na.rm = TRUE
      ),

    `Security or misuse` =
      sum(
        security_misuse,
        na.rm = TRUE
      )
  ) |>
  pivot_longer(
    everything(),
    names_to = "risk",
    values_to = "n_incidents"
  )

```

```

) |>
mutate(
  pct_incidents =
    ifelse(
      n_genai_incidents > 0,
      100 *
        n_incidents /
        n_genai_incidents,
      NA_real_
    )
) |>
arrange(
  desc(n_incidents)
)

```

```

risk_prevalence |>
  rename(
    `Failure pattern` = risk,
    Incidents = n_incidents,
    `GenAI incidents (%)` =
      pct_incidents
  ) |>
  mutate(
    `GenAI incidents (%)` =
      round(
        `GenAI incidents (%)`,
        1
      )
  ) |>
  knitr::kable(
    caption = "Prevalence of selected failure-pattern indicators across identified GenAI inc.
  )

```

Table 18: Prevalence of selected failure-pattern indicators across identified GenAI incidents.

Failure pattern	Incidents	GenAI incidents (%)
Incorrect or fabricated output	329	55
Fraud or impersonation	280	55
Privacy or confidential-data risk	260	55
Security or misuse	191	55
Bias or discrimination	122	55

Failure pattern	Incidents	GenAI incidents (%)
Unsafe or harmful advice	72	55
Automation or oversight failure	72	55

```

risk_prevalence |>
  mutate(
    risk =
      forcats::fct_reorder(
        risk,
        pct_incidents
      )
  ) |>
  ggplot(
    aes(
      x = risk,
      y = pct_incidents
    )
  ) +
  geom_col() +
  coord_flip() +
  labs(
    x = NULL,
    y = "Percentage of identified GenAI incidents"
  ) +
  theme_minimal()

```

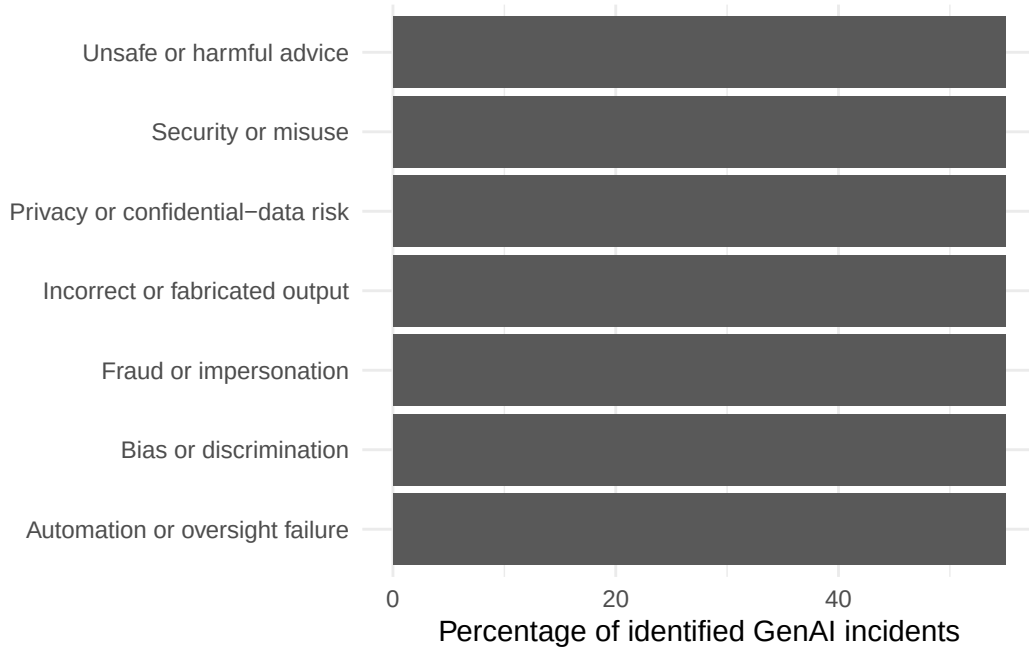


Figure 4: Prevalence of selected failure-pattern indicators across identified generative AI incidents.

The figure provides the main quantitative answer to the first part of the challenge: **which recurring failure patterns appear most often in the identified generative AI incident corpus.**

The percentages describe the share of screened incidents containing evidence matching each indicator. They do not represent the probability that a future SEC Victoria deployment will experience that failure.

8.3 Number of Failure Patterns per Incident

An incident may contain several risks at once. This matters because operational failures are often not caused by a single isolated weakness.

```
genai_risks <- genai_risks |>
  mutate(
    n_failure_patterns =
      rowSums(
        across(
          c(
```

```

        inaccurate_output,
        unsafe_advice,
        privacy_data,
        bias_discrimination,
        fraud_impersonation,
        automation_oversight,
        security_misuse
      )
    ),
    na.rm = TRUE
  )
)

multi_risk_summary <- genai_risks |>
  count(
    n_failure_patterns,
    name = "n_incidents"
  ) |>
  mutate(
    pct_incidents =
      100 *
      n_incidents /
      sum(n_incidents)
  )

knitr::kable(
  multi_risk_summary |>
    rename(
      `Failure patterns identified` =
        n_failure_patterns,
      Incidents = n_incidents,
      `GenAI incidents (%)` =
        pct_incidents
    ) |>
    mutate(
      `GenAI incidents (%)` =
        round(
          `GenAI incidents (%)`,
          1
        )
    ),
  caption = "Number of selected failure patterns identified within each GenAI incident."
)

```

)

Table 19: Number of selected failure patterns identified within each GenAI incident.

Failure patterns identified	Incidents	GenAI incidents (%)
0	54	9.0
1	146	24.4
2	171	28.6
3	118	19.7
4	67	11.2
5	36	6.0
6	6	1.0

```
multi_risk_summary |>
  ggplot(
    aes(
      x = factor(
        n_failure_patterns
      ),
      y = pct_incidents
    )
  ) +
  geom_col() +
  labs(
    x = "Number of failure patterns identified",
    y = "Percentage of identified GenAI incidents"
  ) +
  theme_minimal()
```

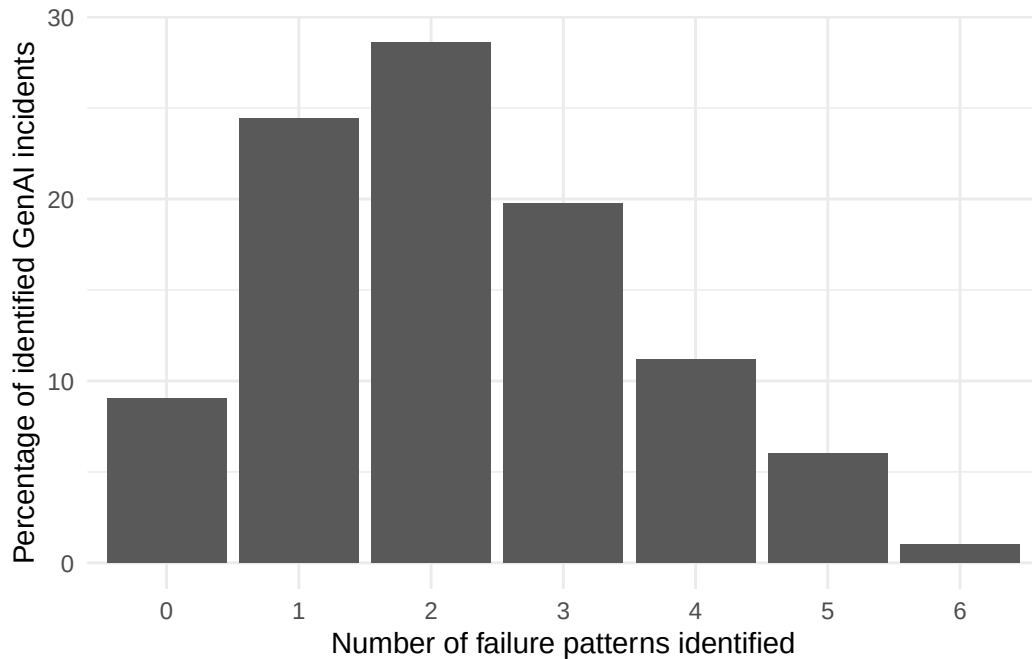


Figure 5: Number of selected failure-pattern indicators identified within each generative AI incident.

This provides evidence for using **layered safeguards** rather than assuming a single control will address every failure mechanism.

9 Root-Cause and Control-Gap Patterns

The incident text can also be used to identify recurring control gaps associated with failures.

These indicators should be interpreted as **observable patterns or proxies**, not definitive causal findings. Incident reports do not provide a controlled experiment capable of proving causation.

9.1 Classify Control-Gap Indicators

```
genai_control_gaps <- genai_risks |>
  mutate(
    verification_gap =
```



```

str_detect(
  analysis_text,
  regex(
    paste(
      c(
        "hallucinat",
        "fabricat",
        "false information",
        "incorrect information",
        "inaccurate information",
        "wrong answer",
        "failed to verify",
        "not verified",
        "unverified"
      ),
      collapse = "|"
    ),
    ignore_case = TRUE
  )
),

human_oversight_gap =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "without human",
          "lack of human",
          "human oversight",
          "lack of oversight",
          "no oversight",
          "human review",
          "automated decision",
          "autonomous decision"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  ),

```

```

data_governance_gap =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "privacy",
          "personal data",
          "personal information",
          "confidential data",
          "sensitive data",
          "data leak",
          "data breach",
          "training data"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  ),

security_control_gap =
  str_detect(
    analysis_text,
    regex(
      paste(
        c(
          "prompt injection",
          "jailbreak",
          "security vulnerability",
          "security flaw",
          "unauthorised access",
          "unauthorized access",
          "cybersecurity"
        ),
        collapse = "|"
      ),
      ignore_case = TRUE
    )
  ),

fairness_governance_gap =

```

```

str_detect(
  analysis_text,
  regex(
    paste(
      c(
        "\\bbias\\b",
        "\\bbiased\\b",
        "discriminat",
        "stereotyp",
        "unequal treatment",
        "unfair treatment"
      ),
      collapse = "|"
    ),
    ignore_case = TRUE
  )
),

misuse_abuse =
str_detect(
  analysis_text,
  regex(
    paste(
      c(
        "\\bfraud\\b",
        "\\bscam\\b",
        "impersonat",
        "deepfake",
        "phishing",
        "malicious use",
        "misuse"
      ),
      collapse = "|"
    ),
    ignore_case = TRUE
  )
)
)

```

9.2 Quantify Control Gaps

```
control_gap_prevalence <- genai_control_gaps |>
  summarise(
    `Verification or grounding gap` =
      sum(
        verification_gap,
        na.rm = TRUE
      ),

    `Human oversight gap` =
      sum(
        human_oversight_gap,
        na.rm = TRUE
      ),

    `Data-governance gap` =
      sum(
        data_governance_gap,
        na.rm = TRUE
      ),

    `Security-control gap` =
      sum(
        security_control_gap,
        na.rm = TRUE
      ),

    `Fairness-governance gap` =
      sum(
        fairness_governance_gap,
        na.rm = TRUE
      ),

    `Misuse or abuse` =
      sum(
        misuse_abuse,
        na.rm = TRUE
      )
  ) |>
  pivot_longer(
    everything(),
```

```

    names_to = "control_gap",
    values_to = "n_incidents"
  ) |>
  mutate(
    pct_incidents =
      ifelse(
        n_genai_incidents > 0,
        100 *
          n_incidents /
          n_genai_incidents,
        NA_real_
      )
  ) |>
  arrange(
    desc(n_incidents)
  )

```

```

control_gap_prevalence |>
  mutate(
    control_gap =
      forcats::fct_reorder(
        control_gap,
        pct_incidents
      )
  ) |>
  ggplot(
    aes(
      x = control_gap,
      y = pct_incidents
    )
  ) +
  geom_col() +
  coord_flip() +
  labs(
    x = NULL,
    y = "Percentage of identified GenAI incidents"
  ) +
  theme_minimal()

```

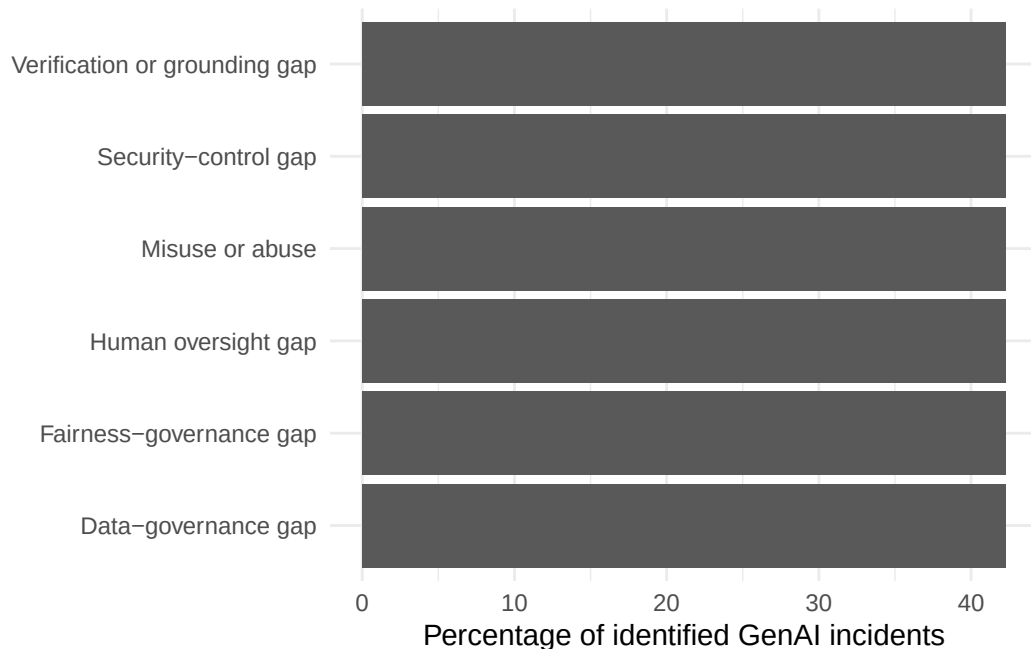


Figure 6: Prevalence of selected control-gap indicators across identified generative AI incidents.

These results help move the analysis from describing **what went wrong** to identifying **which controls should receive the greatest attention** in an internal SEC Victoria rollout.

10 Implications for SEC Victoria

The AIID data do not contain SEC Victoria deployments. The relevance assessment therefore combines the quantified incident patterns with the three operational areas specified in the challenge brief.

The statistical evidence comes from AIID; the operational mapping is an interpretation of how the same failure mechanism could affect SEC Victoria.

10.1 Retail Operations

Internal generative AI could support staff responding to customer enquiries, summarising account information, explaining products or tariffs, drafting communications, or handling internal knowledge queries.

The incident evidence suggests that the most relevant controls are those addressing inaccurate outputs, confidential information, unfair treatment and unchecked reliance on generated recommendations.

```
sec_retail_map <- tibble(
  `AIID failure pattern` = c(
    "Incorrect or fabricated output",
    "Privacy or confidential-data risk",
    "Bias or discrimination",
    "Automation or oversight failure"
  ),
  `Potential SEC retail consequence` = c(
    "Incorrect tariff, account, product or customer-service information",
    "Customer or account information exposed through prompts or generated outputs",
    "Unequal or inappropriate responses across different customer groups",
    "Staff acting on generated information without sufficient verification"
  )
)

knitr::kable(
  sec_retail_map,
  caption = "How recurring GenAI failure patterns could translate to SEC Victoria retail operations"
)
```

Table 20: How recurring GenAI failure patterns could translate to SEC Victoria retail operations.

AIID failure pattern	Potential SEC retail consequence
Incorrect or fabricated output	Incorrect tariff, account, product or customer-service information
Privacy or confidential-data risk	Customer or account information exposed through prompts or generated outputs
Bias or discrimination	Unequal or inappropriate responses across different customer groups
Automation or oversight failure	Staff acting on generated information without sufficient verification

10.2 Wholesale Operations

Wholesale activities can involve commercially sensitive information and higher-consequence analytical decisions.

Generative AI used for summarisation, market intelligence, internal analysis or decision support should therefore be treated differently from low-consequence drafting tasks.

```
sec_wholesale_map <- tibble(
  `AIID failure pattern` = c(
    "Incorrect or fabricated output",
    "Privacy or confidential-data risk",
    "Security or misuse",
    "Automation or oversight failure"
  ),
  `Potential SEC wholesale consequence` = c(
    "Incorrect summaries, unsupported market claims or analytical assumptions",
    "Exposure of commercially sensitive information",
    "Prompt injection, malicious content or inappropriate access to internal information",
    "Generated analysis influencing consequential decisions without independent validation"
  )
)

knitr::kable(
  sec_wholesale_map,
  caption = "How recurring GenAI failure patterns could translate to SEC Victoria wholesale operations"
)
```

Table 21: How recurring GenAI failure patterns could translate to SEC Victoria wholesale operations.

AIID failure pattern	Potential SEC wholesale consequence
Incorrect or fabricated output	Incorrect summaries, unsupported market claims or analytical assumptions
Privacy or confidential-data risk	Exposure of commercially sensitive information
Security or misuse	Prompt injection, malicious content or inappropriate access to internal information
Automation or oversight failure	Generated analysis influencing consequential decisions without independent validation

10.3 Household Electrification

Generative AI could also assist internal teams supporting households considering electrification, such as solar, batteries, electric appliances or other energy upgrades.

The key risk is that generated guidance can sound authoritative even when it is incomplete, unsupported or unsuitable for a particular household.

```
sec_electrification_map <- tibble(
  `AIID failure pattern` = c(
    "Incorrect or fabricated output",
    "Unsafe or harmful advice",
    "Privacy or confidential-data risk",
    "Bias or discrimination"
  ),
  `Potential household-electrification consequence` = c(
    "Incorrect information about technologies, eligibility, costs or installation considerations",
    "Recommendations that are unsuitable or potentially unsafe without qualified assessment",
    "Household, property or customer information disclosed inappropriately",
    "Different households receiving systematically different quality of guidance"
  )
)

knitr::kable(
  sec_electrification_map,
  caption = "How recurring GenAI failure patterns could translate to household-electrification support."
)
```

Table 22: How recurring GenAI failure patterns could translate to household-electrification support.

AIID failure pattern	Potential household-electrification consequence
Incorrect or fabricated output	Incorrect information about technologies, eligibility, costs or installation considerations
Unsafe or harmful advice	Recommendations that are unsuitable or potentially unsafe without qualified assessment
Privacy or confidential-data risk	Household, property or customer information disclosed inappropriately
Bias or discrimination	Different households receiving systematically different quality of guidance

11 Recommended Safeguards

The safeguards below are designed to respond directly to the failure patterns measured in the AIID analysis.

They should be implemented proportionately: low-risk drafting tasks do not require the same controls as tools that influence customer outcomes, commercially sensitive analysis or household energy recommendations.

```
sec_safeguards |>
  transmute(
    `AIID finding` =
      risk,

    `Incidents` =
      n_incidents,

    `GenAI incidents (%)` =
      round(
        pct_incidents,
        1
      ),

    `SEC relevance` =
      sec_application,

    `Recommended safeguard` =
      safeguard
  ) |>
  knitr::kable(
    caption = "Recommended safeguards linked to quantified AIID failure-pattern indicators."
  )
```

Table 23: Recommended safeguards linked to quantified AIID failure-pattern indicators.

AIID finding	GenAI		SEC relevance	Recommended safeguard
	In- ci- dents	inci- dents (%)		
Incorrect or fabricated output	329	55	Retail, wholesale and household electrification	Ground answers in approved SEC information sources, show source references where practical, and require independent verification for consequential outputs.

AIID finding	GenAI		SEC relevance	Recommended safeguard
	In-cidents	inci-dents (%)		
Fraud or impersonation	280	55	Retail communications and internal security	Restrict automated external communications, verify identity and provenance for generated content, and clearly distinguish AI-generated material where appropriate.
Privacy or confidential-data risk	260	55	Retail, wholesale and household electrification	Apply data classification, role-based access controls, approved-model restrictions and prompt/output logging. Prevent sensitive customer or commercial information from entering unapproved models.
Security or misuse	191	55	Wholesale and internal systems	Use access controls, prompt-injection testing, logging and monitoring, and isolate GenAI systems from direct control of critical operational infrastructure.
Bias or discrimination	122	55	Retail and household electrification	Test outputs across relevant customer groups, monitor for materially different treatment, and establish a process for reviewing disputed or harmful outputs.
Unsafe or harmful advice	72	55	Household electrification and customer support	Define prohibited advice categories and escalation rules. Require qualified human review before technical, safety-sensitive or household-specific recommendations are acted upon.
Automation or oversight failure	72	55	Retail, wholesale and household electrification	Keep high-consequence uses advisory-only by default. Require explicit human approval before generated outputs trigger customer, commercial or operational actions.

11.1 Recommended Rollout Model

The incident evidence supports a **risk-tiered rollout** rather than treating every GenAI use case as equally safe.

11.1.1 Tier 1: Low-consequence assistance

Examples include:

- drafting internal text;
- summarising non-sensitive documents;
- brainstorming;
- formatting content.

These uses can operate with lighter controls, although staff should still verify generated content.

11.1.2 Tier 2: Customer or analytical decision support

Examples include:

- customer-service assistance;
- tariff or product explanations;
- household-electrification guidance;
- market or commercial analysis.

These uses should require approved information sources, monitoring, clear source attribution where practical and human review before consequential information is relied upon.

11.1.3 Tier 3: High-consequence or operational actions

Examples include:

- automated customer decisions;
- actions involving commercially sensitive information;
- safety-sensitive recommendations;
- direct interaction with critical operational systems.

The incident patterns support keeping generative AI **advisory rather than autonomous** in these applications unless the system has undergone substantially stronger validation, governance and assurance.

12 Key Findings

The analysis provides four main findings for SEC Victoria.

First, AI incident reporting is highly uneven. Some incidents receive many more reports than others, meaning raw report counts can exaggerate the apparent importance of highly publicised events. The main risk analysis therefore uses distinct incidents as its unit of analysis.

Second, the identified generative AI incidents contain multiple recurring failure patterns rather than one dominant technical problem. These include incorrect or fabricated outputs, privacy and data issues, unsafe advice, unfair outcomes, misuse and insufficient oversight.

Third, several incidents contain more than one selected failure pattern. This supports a layered-control approach in which verification, human oversight, data governance and security operate together rather than as isolated safeguards.

Finally, the implications differ across SEC Victoria’s operations. Customer-facing and household-support tasks create greater risks from incorrect advice, privacy failures and unequal treatment, while wholesale and internal analytical uses place greater emphasis on verification, commercially sensitive information, security and human approval for consequential decisions.

13 Limitations and Ethical Considerations

The results should be interpreted as evidence about **patterns in reported historical incidents**, not as a forecast of the probability that SEC Victoria will experience a particular failure.

AIID is an incident database rather than a random sample of all deployed AI systems. Events that receive media, regulatory or public attention may be more likely to appear in the database. Consequently, the prevalence values in this report describe the selected incident corpus rather than population-wide failure rates.

The generative AI screen is based on identifiable terminology and system names. It may miss incidents where a generative model is not clearly described.

The failure-pattern and control-gap classifications are also based on transparent keyword and phrase indicators. They support reproducible pattern analysis, but they should not be interpreted as definitive causal labels.

Reports in multiple languages are retained. However, the current token cleaning, custom stop words and risk dictionaries are mainly English-focused. Multilingual incidents may therefore be classified less completely.

Some incident categories overlap. An incident involving a privacy failure can also involve weak oversight, misuse or security weaknesses. Percentages across categories therefore should not be summed to 100%.

Publication dates identify when source reports were published and may differ from the date on which an incident occurred.

Despite these limitations, the incident-level approach provides a defensible way to use historical evidence to identify controls that SEC Victoria should prioritise before deploying internal generative AI into higher-consequence workflows.

14 Conclusion

The AI Incident Database suggests that the main challenge for SEC Victoria is not simply whether generative AI can produce useful outputs, but whether those outputs can be trusted and governed when they interact with customers, commercially sensitive information and household energy decisions.

The recurring incident patterns support four broad priorities: verify generated information against trusted SEC sources, retain meaningful human oversight, restrict the use of sensitive information, and apply stronger controls as the consequence of an AI-assisted decision increases.

For SEC Victoria, this means generative AI is best introduced through a risk-tiered model. Low-consequence internal assistance can be adopted with relatively light controls, while retail advice, wholesale analysis and household-electrification recommendations should require stronger verification and oversight. Generative AI should not directly control high-consequence operational actions unless substantially stronger assurance has been demonstrated. ““